

2025年度
修士論文

SN開発におけるデバッグシステムの開発

指導教員	アサノ デービッド	教授
	不破 泰	肩書き

2025年1月30日提出

信州大学大学院 総合理工学研究科 工学専攻 情報数理・融合システム分野
24W6047A
辻村篤志

あらまし

あらましを書く。

目次

第1章	はじめに	1
1.1	研究背景	1
1.2	当研究室における先行研究	2
1.3	先行研究の課題	2
1.4	本研究の目的	2
1.5	本論文の構成	2
第2章	先行研究の提案システム	4
2.1	システムの概要	4
2.2	実現事項	5
2.3	課題	5
第3章	提案システムの概要と設計方針	6
3.1	設計方針	6
3.2	システムの構成	7
3.3	各コンポーネントの概要	7
3.3.1	コード生成コンポーネント	8
3.3.2	組込みシステムコンポーネント	8
3.3.3	デバッグ支援システムコンポーネント	9
第4章	設計及び実装	10
4.1	使用ソフトウェア、ツール	10
4.2	各コンポーネントの接続	10
4.2.1	コード生成コンポーネント	11
4.3	デバッグシステムの設計	11

4.4	ログ取得方式の設計	12
4.5	ログの解析	12
4.6	状態遷移可視化の設計	12
4.7	実装概要	12
第5章	操作例	13
5.1	操作例の概要	13
5.2	操作対象および前提条件	13
5.3	状態遷移図の読み込み	14
5.4	ログ取得と状態遷移の可視化	14
5.5	ステップ実行による逐次確認	14
5.6	変数ウォッチ機能の利用	14
5.7	複数ノード同期デバッグ	15
5.8	操作例のまとめ	15
第6章	評価と考察	16
6.1	評価方針	16
6.2	評価方法	16
6.3	デバッグ効率の評価	16
6.4	状態遷移理解の容易性に関する評価	17
6.5	複数ノード同期デバッグの評価	17
6.6	考察	17
6.7	評価のまとめ	17
第7章	まとめと今後の展望	18
7.1	まとめ	18
7.2	今後の展望	18
	参考文献	19

図 目 次

2.1	先行研究システムの概要図	4
3.1	従来のデバッグ方法	7
3.2	提案システムの概要図	8
3.3	どこでモデム (左) と GPS モジュール (右)	9

表 目 次

第 1 章

はじめに

はじめにを書く。

1.1 研究背景

近年、無線センサネットワーク（Wireless Sensor Network：WSN）や IoT（Internet of Things）は、環境モニタリングやスマートシティ、インフラ監視、農業、ヘルスケアなど、さまざまな分野での応用が期待されている。実際、IoT 全体の接続デバイス数は急速に増加している。IoT Analytics によれば、2023 年時点で約 166 億台、2024 年には約 188 億台に達し、2030 年には約 400 億台に到達すると予測されている [1]。また、WSN の普及も市場規模の面から拡大を続けており、Grand View Research の報告では、産業用 WSN（Industrial WSN）の市場は 2023 年に約 51.9 億ドルと推定され、2024 年から 2030 年にかけて年平均 12.1%の成長が見込まれている [2]。

しかし、これらのシステムを構築するためには、複雑な通信プロトコルの設計やデータ処理の設計および実装が求められ、開発者には幅広い専門知識と多大な工数が必要となる。さらに、無線通信環境の構築やマイコン設定など導入時のハードルも高く、これらの研究や開発の多くはシミュレーション環境やエミュレータ上での検証にとどまることが多い。しかし、シミュレーション環境では実機特有の挙動や外部環境の影響を完全に再現することが難しく、実機での検証が不可欠である場合も多く、実機でのプロトコル検証を容易にするための支援環境の整備が求められている。

1.2 当研究室における先行研究

当研究室の先行研究では、状態遷移図上で無線通信プロトコルの動作を定義し、そこから自動生成したプログラムを汎用ハード上で動作させ、プロトコルを実機で動作検証できる環境が構築されていた（旭ら [3]、小林ら [4]）。

1.3 先行研究の課題

このシステムにより基本的な通信プロトコルの動作検証が可能となったが、デバッグ方法はコンソールへのログ出力に依存しており、通信処理が高速に進むため逐次的な状態遷移を追跡することが難しかった。その結果、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらく、開発効率や教育的利用の観点では十分でない点が課題として残されていた。

1.4 本研究の目的

本研究の目的は、先行研究で課題となっていたデバッグ効率の低さを改善し、実機でのプロトコル検証をより効果的に行える環境を実現することである。そのために、無線通信デバイス上で実行された動作をログとして出力し、それを可視化・ステップ実行できる仕組みを開発した。これにより、プロトコルの動作を逐次的に把握でき、異常動作の原因究明を容易にするとともに、教育やチーム開発、応用実証に適した支援環境を提供することを目指す。

1.5 本論文の構成

本論文は全7章から構成されている。第1章では、本研究の背景として当研究室におけるこれまでの研究の流れを整理し、先行研究で顕在化した課題を明らかにした上で、本研究の目的を述べる。第2章では、当研究室における先行研究システムについて、その構成や実現されてきた機能を整理するとともに、運用およびデバッグの観点から課題をまとめる。第3章では、先行研究の課題を踏まえ、本研究で提案するシステムの概要と設計方針について述べる。ここでは、全体構成および各コンポーネントの役割を中心に説

明する。第4章では、提案システムの設計および実装について述べ、特にデバッグ支援を目的としたログ取得方式や状態遷移の可視化手法について詳述する。第5章では、提案システムの操作例を示し、実際の利用を通してどのような情報が得られるかを説明する。第6章では、従来手法との比較を通じて提案システムの有効性を評価し、その結果について考察を行う。最後に第7章では、本研究のまとめを行うとともに、今後の課題および展望について述べる。

第 2 章

先行研究の提案システム

2.1 システムの概要

先行研究では、無線通信プロトコルの設計・実装・動作検証を支援するシステムが提案されてきた。このシステムは主に次の二つのコンポーネントから構成されている。第一に、UML の状態遷移図を用いてプロトコルの動作を定義し、そこからコードを自動生成するコンポーネントである。第二に、生成されたコードを実行するための環境（ハードウェア）を提供するコンポーネントである。これにより、状態遷移図で定義されたプロトコルが実機上で動作し、その動作を検証できる環境が提供されてきた。システムの概要を図 2.1 に示す。

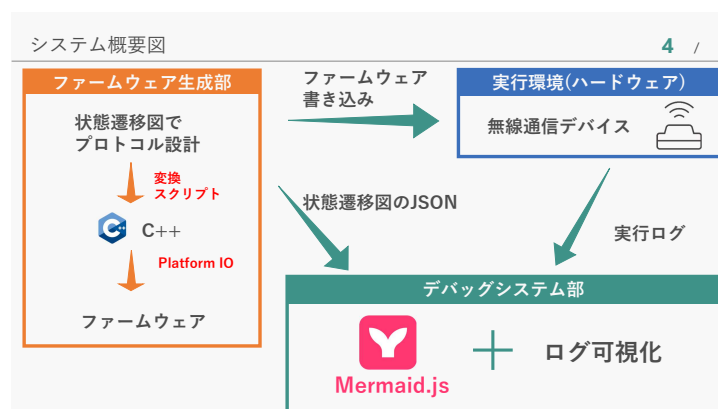


図 2.1: 先行研究システムの概要図

2.2 実現事項

先行研究システムでは、以下の主要な機能が実現されている。

- 状態遷移図からのコード自動生成：UML の状態遷移図を用いてプロトコルの動作を定義し、そこから C 言語コードを自動生成する機能が提供されている。これにより、プロトコル設計者は視覚的に動作を定義でき、手動でのコーディング作業を削減できる。
- 実機上での動作検証：生成されたコードは、無線モデム搭載のマイコンを組み合わせたハードウェア上で実行される。これにより、シミュレーション環境では捉えきれない実機特有の挙動を検証できる。

2.3 課題

先行研究システムには以下の課題が存在する。

- 状態遷移図の記法の制約：先行研究における状態遷移図からのコード自動生成は、特定の記法に依存しており、複雑なプロトコル動作を表現する際に制約がある。これにより、設計者が意図する動作を完全に反映できない場合がある。
- デバッグ効率の低さ：デバッグ方法がコンソールへのログ出力に依存しており、通信処理が高速に進むため逐次的な状態遷移を追跡することが難しい。これにより、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらい。

これらの課題を解決するために、本研究ではデバッグ効率の向上に焦点を当て、実機でのプロトコル検証をより効果的に行える環境を提案する。なお、状態遷移図からのコード生成部については、[5] で改善が図られているため、そちらを参照されたい（たびたびそちらの論文を参照するように指示することがある）。

第 3 章

提案システムの概要と設計方針

本研究では、先行研究システムにおいて課題となっていたデバッグ効率の低さを改善することを目的として、新たなデバッグ支援システムを提案する。本章では、提案システムの概要と設計方針について述べる。はじめに、本研究で対象とする課題を整理し、次に設計方針およびシステム全体の構成について説明する。

3.1 設計方針

先行研究システムでは、状態遷移図を用いて通信プロトコルを設計し、実機上で動作検証を行うことが可能であった。一方で、デバッグ方法は主に図 3.1 のように、コンソールへのログ出力に依存しており、状態遷移の流れや内部動作を逐次的に把握することが難しいという課題があった。

本研究では、この課題を踏まえ、実機上で実行される通信プロトコルの内部動作をより直感的に把握できるデバッグ支援環境の構築を設計方針とした。具体的には、プロトコルの動作を状態遷移として捉え、その実行状況をログとして取得・可視化することで、動作の流れを段階的に確認できる仕組みを目指す。

また、本研究のシステムは、先行研究で構築されたプロトコル設計および実機検証の枠組みを活用することを前提とし、既存システムとの親和性を保ちながら改善を行うことを重視した。これにより、研究室内での継続的な利用や拡張が容易な構成とすることを設計上の方針とした。

```
STATE:IDLE
current_slot = 0
STATE:RECEIVE_SLOT_INFO
STATE:CHECK_RECEIVED_PACKET
STATE:WAIT_OWN_SLOT
current_slot = 1
STATE:WAIT_OWN_SLOT
STATE:WAIT_OWN_SLOT
current_slot = 2
STATE:WAIT_RANDOM_DELAY
STATE:CREATE_PACKET
STATE:TRANSMIT
.
.
.
```

図 3.1: 従来のデバッグ方法

3.2 システムの構成

システム全体の構成を図 3.2 に示す。

提案システムは、先行研究システムに対してデバッグ支援機能を付加する形で構成されている。全体としては、通信プロトコルを実行する組込みシステムと、その動作を外部から確認するためのデバッグ支援システムからなる。

組込みシステム側では、状態遷移に基づいて通信プロトコルが実行され、その実行過程に関する情報がログとして出力される。一方、デバッグ支援システム側では、出力されたログを収集し、状態遷移の流れや実行状況を可視化する役割を担う。

3.3 各コンポーネントの概要

提案システムは、主に以下のコンポーネントから構成されている。

- 状態遷移図からコードを自動生成するコンポーネント

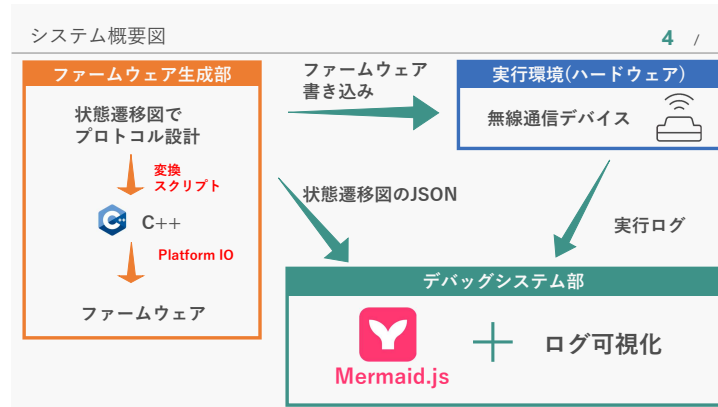


図 3.2: 提案システムの概要図

- 通信プロトコルを実行する組込みシステム
- デバッグ支援システム

3.3.1 コード生成コンポーネント

はじめに、状態遷移図からコードを自動生成するコンポーネントである。このコンポーネントは、UML の状態遷移図を解析し、通信プロトコルの動作を表現する C++コードを生成する役割を担う。これにより、設計者は視覚的にプロトコルの動作を定義でき、手動でのコーディング作業を削減できる。本コンポーネントでは、状態遷移図の構造を解析し、存在する状態、遷移、イベントおよびアクションを抽出する。抽出した情報に基づいて通信プロトコルの動作を表現する C++コードを生成し、状態遷移は switch-case 文を用いて実装される。生成されたコードは PlatformIO を用いてビルドされ、無線通信デバイスに書き込まれた後、組込みシステム上で実行される。

3.3.2 組込みシステムコンポーネント

本システムにおける実行環境を図 3.3 に示す。本研究では、SAM D21 マイコンと無線通信モジュールを一体化した無線通信デバイス (株式会社サーキットデザイン製どこでモデム) に GPS モジュールを組み合わせて、一つの無線通信デバイスとして使用した。この無線通信モジュールは 429MHz 帯で動作する汎用的なものであり、技術基準適合証明 (技



図 3.3: どこでモデム (左) と GPS モジュール (右)

適)を取得しているため、実際に電波を飛ばしながら通信プロトコルの検証を行うことができる。また、429MHz帯は中山間地域において回折性能に優れており、低消費電力で長距離通信が可能なLPWA (Low Power Wide Area) 通信に適している。この特性を活かすことで、登山者見守りシステムなどへの応用も期待できる。さらに、本デバイスはマイコンと通信モジュールが一体化しているため、外部配線や追加ハードウェアを必要とせず、開発や実験環境の構築を容易に行うことが可能である。加えて、GPSモジュールを併用することで、位置情報の取得および1PPS信号による高精度な時刻同期を実現し、複数ノード間での時刻同期による無線通信プロトコルの実現を可能としている。GPSモジュールは使わない場合は外すこともできる。

3.3.3 デバッグ支援システムコンポーネント

最後に、本研究の中核であるデバッグ支援システムである。このコンポーネントは、組み込みシステムから出力されるログを収集し、状態遷移や内部動作を可視化する役割を担う。これにより、従来は把握が困難であったプロトコルの動作過程を確認できる。このコンポーネントは、次章で詳述する。

第 4 章

設計及び実装

本章では、提案システムの設計および実装について述べる。特に、デバッグ支援システムの設計方針、ログ取得方式、および状態遷移可視化の手法について詳述する。

4.1 使用ソフトウェア、ツール

使用ソフトウェア、ツールについて説明する。本研究では、UML 設計ツールとして astah Professional を使用し、状態遷移図の作成を行った。コード編集および開発環境としては Visual Studio Code を使用し、組込みシステムのビルド環境には PlatformIO を用いた。また、デバッグ支援システムのフロントエンド開発には Node Package Manager (npm) を用いて必要なライブラリを管理した。

4.2 各コンポーネントの接続

各コンポーネント間の接続関係について説明する。はじめに、コード生成コンポーネントでは Python スクリプトを実行し、作成した astah の状態遷移図ファイルを入力として C++コードを生成する。この際、デバッグ支援システムで使用するための状態遷移図 JSON ファイルも同時に生成される。生成された C++コードは PlatformIO を用いてビルドされ、無線通信デバイスに書き込まれる。組込みシステム上で実行された通信プロトコルは、動作中の状態遷移や内部変数の情報をログとしてシリアル通信を通じて出力する。これらのログはデバッグ支援システムによって受信され、解析および可視化に利

用される。また、コード生成コンポーネントによって生成された状態遷移図 JSON ファイルは、デバッグ支援システムにアップロードされ、状態遷移の可視化に用いられる。

4.2.1 コード生成コンポーネント

コード生成コンポーネントは、UML の状態遷移図を解析し、通信プロトコルの動作を表現するプログラムコードを自動生成する役割を担う。本研究では、このコンポーネントとして共同研究（小林侑生ら）により開発された手法を利用している。本コンポーネントでは、状態遷移図の構造を解析し、存在する状態、遷移、イベントおよびアクションを抽出する。抽出した情報に基づき、通信プロトコルの動作を表現する C++コードを生成し、状態遷移は switch-case 文を用いて実装される。生成されたコードは PlatformIO を用いてビルドされ、無線通信デバイスに書き込まれた後、組み込みシステム上で実行される。なお、本研究の主眼はコード生成手法そのものではなく、生成されたコードを用いた実機デバッグ支援環境の構築にあるため、コード生成手法の詳細については文献 [5] を参照されたい。

4.3 デバッグシステムの設計

本研究で提案するデバッグ支援システムは、ブラウザ上で動作するフロントエンドアプリケーションとして実装されている。UI およびロジックの構築には React.js を用いた。本システムは、単一ノードのログを解析するログモードと、2 台のノードを同期させて解析する同期モードの 2 つの動作モードを備えている。ログモードでは、ユーザが状態遷移図の JSON ファイルをアップロードし、組み込みシステムから送信されるログをリアルタイムで受信・解析する。解析されたログは状態遷移および各ステップにおける変数値として整理され、状態遷移図上に現在の状態がハイライト表示される。同期モードでは、デバッグ支援システム上のシリアルモニタ機能を用いて、2 台の無線通信デバイスからのログを同時に受信する。受信したログを基に同期タイムラインを生成し、両デバイスの状態遷移および変数値を同一の時間軸上で比較することが可能である。また、本システムではステップ実行機能を備えており、ユーザは状態遷移を一つずつ確認することができる。さらに、複数の変数を同時に監視できるウォッチ機能や、スライダー操作による任意ステップへのジャンプ機能も実装している。

4.4 ログ取得方式の設計

4.5 ログの解析

- 組込みシステムから送信されるログは、特定のフォーマットに従っている- ログメッセージは、状態遷移イベント、変数の変更、デバッグ情報などを含む

4.6 状態遷移可視化の設計

状態遷移の可視化には、astah で作成した状態遷移図を JSON 形式でエクスポートしたものを使用する。ユーザはデバッグ支援システム上で JSON ファイルをアップロードし、Mermaid.js を用いて状態遷移図を描画する。組込みシステムから取得したログと連携することで、状態遷移図上に現在の状態をハイライト表示し、状態遷移の進行を視覚的に確認できる。

4.7 実装概要

実装概要を書く。

第 5 章

操作例

5.1 操作例の概要

本章では、提案システムの具体的な操作手順を示し、無線通信プロトコルのデバッグ作業においてどのような情報が得られるかを明らかにする。対象とする利用シナリオは、実機上で動作する通信プロトコルの挙動確認および不具合解析であり、従来のコンソールログベースのデバッグ手法と比較して、提案システムによって得られる利点を示すことを目的とする。特に、本章では教育的観点から、状態遷移の可視化およびステップ実行機能の有効性に着目する。

5.2 操作対象および前提条件

操作例では、状態遷移図に基づいて生成された MAC プロトコルを対象とする。本研究では、教育的に理解しやすい基本的な通信方式である pure ALOHA 方式をベースとした簡易的な MAC プロトコルを用いる。pure ALOHA は状態数および遷移が比較的少なく、状態遷移に基づく動作理解が容易であるため、本研究におけるデバッグ支援機能の説明に適している。

本プロトコルの状態遷移図は UML 設計ツールで作成し、JSON 形式で出力したものをを用いる。無線通信デバイスは 2 台構成とし、それぞれを独立したノードとして動作させる。各ノードからは、状態遷移および内部変数の変化を含むログがシリアル通信を通じて出力される。

以下では、まず単一ノードを対象としたログモードでの操作例を示し、続いて複数ノードを対象とした同期モードでの操作例について説明する。

5.3 状態遷移図の読み込み

はじめに、デバッグ支援システムを起動し、対象とする状態遷移図の JSON ファイルをアップロードする。アップロードされた状態遷移図は Mermaid.js を用いて描画され、状態および遷移の関係を視覚的に確認できる。これにより、プロトコル全体の構造を直感的に把握することが可能となる。

5.4 ログ取得と状態遷移の可視化

次に、無線通信デバイスを起動し、シリアル通信を通じてログを取得する。取得されたログはデバッグ支援システム内部で解析され、状態遷移および各ステップにおける変数値として整理される。解析結果は状態遷移図上に反映され、現在の状態がハイライト表示される。ログの進行に応じてハイライト表示が更新されるため、状態遷移の流れを視覚的に追跡できる。

5.5 ステップ実行による逐次確認

提案システムでは、取得したログをステップ単位で再生する機能を備えている。ユーザは操作ボタンを用いて状態遷移を一つずつ進めたり、前の状態に戻したりすることができる。また、スライダー操作により任意のステップへ直接移動することも可能である。これにより、通信処理が高速に進行する場合であっても、状態遷移を逐次的に確認できる。

5.6 変数ウォッチ機能の利用

状態遷移の確認に加え、ユーザは特定の内部変数を選択して監視することができる。複数の変数を同時に表示することが可能であり、状態遷移と変数値の変化の関係を同時に把握できる。これにより、プロトコル内部の動作をより詳細に確認できる。

5.7 複数ノード同期デバッグ

次に、複数ノードを対象とした同期モードでの操作例を示す。同期モードでは、デバッグ支援システム上のシリアルモニタ機能を用いて、2台の無線通信デバイスからのログを同時に取得する。取得したログを基に、デバッグ支援システム内で同期タイムラインを生成し、両ノードの状態遷移および変数値を同一の時間軸上で比較する。なお、本同期処理はGPS等による時刻同期を用いず、ログの取得順序および内部タイムスタンプに基づいて行われる。

5.8 操作例のまとめ

以上の操作例より、提案システムを用いることで、状態遷移の流れを視覚的に把握できること、ステップ実行によって逐次的な確認が可能であること、および複数ノード間の動作関係を同時に確認できることを示した。

第 6 章

評価と考察

6.1 評価方針

本章では、提案システムの有効性を定性的に評価することを目的とする。評価は、従来のコンソールログベースのデバッグ手法と比較することで行い、特にデバッグ効率および状態遷移の理解容易性の観点から検討する。

6.2 評価方法

評価対象として、同一の通信プロトコルおよび同一の不具合を用意し、従来手法と提案手法の2通りでデバッグを実施した。本研究では、実際のデバッグ作業を通じて得られた知見を基に、操作性や理解のしやすさといった観点から定性的な比較を行う。

6.3 デバッグ効率の評価

従来手法では、ログが高速に出力されるため、状態遷移の対応関係を把握するためにログを何度も確認する必要があった。一方、提案手法では、状態遷移の可視化およびステップ実行機能により、動作を逐次的に確認することが可能となり、不具合の原因を把握するまでの作業負荷が軽減された。

6.4 状態遷移理解の容易性に関する評価

従来手法では、状態遷移の流れをログから読み取り、利用者が頭の中で再構成する必要があった。これに対し、提案手法では現在の状態が常に状態遷移図上に表示されるため、状態遷移の流れを直感的に把握できた。

6.5 複数ノード同期デバッグの評価

複数ノードを用いた場合、従来手法ではノードごとのログを個別に確認し、手動で突き合わせる必要があった。提案手法では、同期タイムライン上で両ノードの状態遷移を同時に確認できるため、ノード間の状態ずれや動作の不整合を容易に把握できた。

6.6 考察

以上の評価結果より、提案システムは、複雑な状態遷移を含む通信プロトコルや、複数ノードが関与する通信において特に有効であることが示された。一方で、本評価は定性的な観点に基づくものであり、今後は定量的指標を導入した評価手法の検討が課題として挙げられる。

6.7 評価のまとめ

本章では、提案システムがデバッグ効率の向上および状態遷移理解の容易化に有効であることを定性的に示した。

第 7 章

まとめと今後の展望

7.1 まとめ

本研究の成果と意義を総括する。本研究の新規性は以下の 3 点にある。1. 実機ログを用いた状態遷移の逐次可視化 2. ステップ実行可能なデバッグ支援 3. 複数ノードの同期タイムラインによる比較分析

7.2 今後の展望

今後の課題と研究の方向性について述べる。

参考文献

- [1] IoT Analytics: "State of IoT 2024: Number of Connected IoT Devices Grows 16% to 16.6 Billion", 2024.
- [2] Grand View Research: "Industrial Wireless Sensor Network Market Size, Share & Trends Analysis Report, 2024–2030", 2024.
- [3] 旭 健汰, アサノ デービッド, 不破 泰: "無線モデムを用いたセンサネットワーク MAC プロトコル検証システムの開発", 信学技報, NS2023-147, pp.120–125, Dec. 2023.
- [4] 小林 遼, アサノ デービッド, 不破 泰: "センサーネットワーク検証システムにおける遷移図を用いた MAC プロトコル設計環境の開発", 信学技報, NS2023-148, pp.126–131, Dec. 2023.
- [5] 小林 侑生, アサノ デービッド, 不破 泰:
- [6] PlatformIO Labs, "What is PlatformIO?", <https://docs.platformio.org/en/latest/what-is-platformio.html>, 参照 Oct. 15, 2025.
- [7] Mermaid.js, "Overview," <https://mermaid.js.org/intro/>, 参照 Oct. 15, 2025.
- [8] Mermaid.js, "State Diagram Syntax (stateDiagram-v2)," <https://mermaid.js.org/syntax/stateDiagram.html>, 参照 Oct. 15, 2025.
- [9] Eclipse Foundation, "MQTT: Message Queuing Telemetry Transport," <https://mqtt.org/>, 参照 Oct. 15, 2025.
- [10] 園田 継一郎, 不破 泰, アサノ デービッド, "無線センサーネットワークの端末・中継機における送信タイミング決定時間短縮方法の検討," 信学技報, NS2022-138, Dec. 2022.